



POLITECNICO
MILANO 1863

COMPUTER SCIENCE AND ENGINEERING
SOFTWARE ENGINEERING II
2024- 2025

Done By:

Nafis Ismam - 11117431

Ayman Mansur - 11119831

Chris Khoury - 11085985

**Requirement Analysis and Specification
Document**

Deliverable: RASD

Title: Requirement Analysis and Verification Document

Authors: Nafis Ismam, Ayman Mansur, Chris Khoury

Version: 2.0

Date: 23 December 2025

Download page: <https://github.com/chriskhoury1/KhouryMansurIsmam>

Copyright: Copyright © 2017, YOUR NAMES – All rights reserved

Contents

Table of Contents.....	3
List of Figures.....	4
List of Tables.....	4
1 Introduction.....	5
1.1 Purpose	
1.2 Scope	
1.3 Definitions, Acronyms, and Abbreviations	
1.4 Revision History	
1.5 Reference Documents	
1.6 Document Structure	
2 Overall Description.....	9
2.1 Product Perspective	
2.2 Product Functions	
2.3 User Characteristics	
2.4 Assumptions, Dependencies, and Constraints	
3 Specific Requirements.....	23
3.1 External Interface Requirements	
3.2 Functional Requirements	
3.3 Performance Requirements	
3.4 Design Constraints	
3.5 Software System Attributes	
4 Formal Analysis Using Alloy.....	36
5 Effort Spent.....	42
References.....	43

List of Figures

- 2.1 Domain Class Diagram
- 2.1 State Machines 1 to 4
- 3.1 Use case diagram Best Bike Paths
- 3.2 Record trip and optional weather
- 3.3 Manual insert bike path information
- 3.4 Automated insert and detect obstacles
- 3.5 Confirm and Correct obstacles
- 3.6 Search and visualize paths
- 3.7 Merge Information
- 4 Initial Alloy Model
- 4 Examples 1 and 2
- 4 Assertion 1 and 2

List of Tables

- 1.4 Revision History
- 5 Effort Spent

1. Introduction

1.1 Purpose

The purpose of this Requirement Analysis and Specification for the Best Bike Paths(BBP) software system is to define the high-level goals of the project, its functionalities, and the constraints under which it must operate. The primary objective of the BBP project is to develop a software system that supports a community of cyclists in creating, maintaining, and browsing a high-quality inventory of bike paths. The system is intended to enhance the cycling experience by providing reliable tools for navigation, route planning, and data contribution.

To achieve this feat, the BBP system will fulfill these key objectives.

- Provide a search and visualization tool for all users to find and evaluate bike paths between a specific origin and destination. Paths will be ranked using a “Path Score” that reflects both user-sourced quality and route efficiency and effectiveness.
- Offer a personal dashboard for registered users to record, store, and analyze their cycling trips. This includes performance metrics, such as distance and speed, as well as, when possible, external meteorological data.
- For Data Contribution, it will enable registered users to add and update information about bike paths. Two methods will support this:
 1. Manual mode: For users to directly input details, such as street names and path status.
 2. Automated mode: It will intelligently capture data via GPS and potential obstacles, such as potholes, using the gyroscope or accelerometer on the user’s mobile device, subject to user confirmation before publishing.

Implement a backend system for data reliability that merges information from multiple users to present a consolidated, reliable, and up-to-date view of path conditions, taking into account data freshness and the number of confirmations.

1.2 Scope

This document analyzes the BPP system, the World Only and Shared phenomena, where the Shared phenomena have two subdivisions: World Controlled and Machine Controlled, between the system and the environment.

1.2.1 World Phenomena

These are elements or conditions that exist in the environment, independent of the BBP system's operation.

1. The existence of **Users** (both Unregistered and Registered).
2. The existence of **Mobile Devices** (smartphones, tablets) and their hardware-level sensors (GPS, accelerometer, gyroscope).
3. The existence of External Digital Services, such as the **external meteorological service** and the **external map/routing service**.
4. The Physical Domain, including the real-world network of roads, dedicated bike tracks, and their physical conditions (e.g., potholes, surfaces).

1.2.2 Shared Phenomena

These are the interactions and data exchanges that occur at the boundary between the "world" and the BBP system ("machine").

- World Controlled Phenomena
 1. Events or data initiated by the world and received by the system.
 2. A user submits a search query, provides manual path data, or confirms/corrects automated data.
 3. The BBP application receives the stream of GPS, accelerometer, and gyroscope data generated by the user's mobile device.
 4. The BBP system receives a response (e.g., a JSON payload with weather data) from an external service.

- Machine Controlled Phenomena
 1. Events or data initiated by the system and observed by the world.
 2. The BBP system displays search results on a map, presents trip statistics, or prompts a user for confirmation.
 3. The BBP system sends a request (e.g., for weather at a specific location) to an external service.

1.3 Definitions, Acronyms, and Abbreviations

This section clarifies the terminology used throughout the documents to ensure an unambiguous understanding of the document.

1. **BBP:** Best Bike Paths (the software system being specified).
2. **Bike Path:** A route suitable for cycling. As per the problem description, this includes either a proper bike track or a path where cars are rare and speed limits are compatible with cycling.
3. **False Positive:** An event where the automated mode incorrectly identifies an obstacle (e.g., a pothole) that does not exist.
4. **GPS:** Global Positioning System.
5. **Obstacle:** A physical problem on a path that degrades its quality, such as a pothole.
6. **Path Score:** A computed value used to rank paths. It is based on the Path Status and the effectiveness of the path for the requested trip.
7. **Path Status:** A user-provided rating of a path's condition (e.g., optimal, medium, sufficient, requires maintenance).
8. **RASD:** Requirements Analysis and Specification Document.
9. **Registered User:** A user who has created an account, granting access to trip recording and data contribution features.
10. **UML:** Unified Modeling Language.
11. **Unregistered User:** A user interacting with the system without an account, limited to browsing and searching for paths.

1.4 Revision History

This table tracks the document's history and the changes made over time.

Version	Date	Author(s)	Summary
1.0	12/11/2025	Nafis Ismam	Introduction and Initial Draft.
1.1	14/11/2025	Ayman Mansur	First draft of Specific Requirements.
1.2	21/11/2025	Ayman Mansur	Final draft of Specific Requirements.
1.3	28/11/2025	Chris Khoury	Overall Description and State Machines drafted.
1.4	05/12/2025	Nafis Ismam	Alloy formal model drafted and initial verification.
1.5	10/12/2025	Chris Khoury	Internal review and feedback integration.

1.6	15/12/2025	Nafis Ismam	Alloy model execution and results added.
1.7	18/12/2025	Ayman Mansur	Diagrams refined and sequence diagrams updated.
1.8	20/12/2025	Nafis Ismam	Merged contributions and edited for consistency.
2.0	23/12/2025	Nafis Ismam, Ayman Mansur, Chris Khoury	Final document compilation, formatting, and submission.

1.5 Reference Documents

1. The preparation of this Requirement Analysis and Specification Document (RASD) for the Best Bike Paths (BBP) project has been guided by the following reference documentation. These sources define the structure, quality goals, standards, and expectations for requirements engineering artifacts within the context of the Software Engineering II course and general best practices in requirements engineering:
2. Software Engineering II (SE2) Project Assignment Requirement Engineering and Design Project Specification, Politecnico di Milano, A.Y. 2024/2025. The official project assignment sheet defines the problem to be addressed and the expected deliverables for the RASD and subsequent documents, including DD, as provided by the course.
3. Course slides and lecture notes on Requirements Engineering and RASD Structure, provided on the SE2 course webpage (Politecnico di Milano). These materials explain the recommended document structure, definition of phenomena, functional and non-functional requirement specification, and overall quality goals for RASD artifacts.
4. ISO/IEC/IEEE 29148:2018 Systems and Software Engineering Life Cycle Processes Requirements Engineering, International Standard. This standard defines best practices, processes, recommended content, and quality attributes for requirements engineering and the construction of requirements specification documents.
5. ISO/IEC/IEEE 29148:2018 Requirements Engineering Standard, IEEE Xplore (official IEEE standard repository). Provides authoritative descriptions of requirements engineering processes and structures that support the documentation and verification of requirements.
6. "Requirements Structure for System Requirements Formal Modelling, Verification and Validation," IFAC-PapersOnLine, 2024 discusses how formal modelling techniques like Alloy can be used to improve early verification and validation of requirement specifications.
7. "Requirements Engineering for Sustainable Software Systems: A Systematic Mapping Study," Requirements Engineering, Springer, 2025, provides insight into modern requirements engineering methods, documentation practices, and classification of requirement activities useful to the quality of RASD artifacts.
8. Visure Solutions, "Che cosa è Requirements Engineering: Processo per software e sistemi," Visure Solutions guide outlines the core activities and best practices for requirements elicitation, analysis, documentation, and management, supporting the structured definition of requirements.
9. Studocu – Software Engineering 2 RASD and DD Example Resources, including structure and

typical content requirements for SE2 projects, including RASD artefacts.

10. Politecnico di Milano – SE2 Course Materials (lecture notes on RE process) that explain requirements engineering processes, cost of errors due to poor requirements, elicitation, and documentation practices.

1.6 Document Structure

This RASD document is organized into six main sections, following the structure provided for the course project.

1. Introduction: This section outlines the document's objectives, purpose, and scope through an analysis of the systems world and shared phenomena. It also contains the definitions, revision history, reference documents, and an overview of the document's structure.
2. Overall Description: This section provides a high-level overview of the BBP system. It details the product perspective, which includes scenarios, further details on the shared phenomena, the domain models, and diagrams. It also summarizes the main product functions, which are the most important requirements, describes user characteristics, and lists key domain assumptions, dependencies, and constraints.
3. Specific Requirements: This section will elaborate on the topics mentioned in Section 2, providing the details necessary for the development team. It is categorized into:
 - External Interface Requirements.
 - Functional Requirements: Defined using diagrams, such as Use case diagrams, Activity or Sequence diagrams, and their mapping into requirements.
 - Performance Requirements.
 - Design Constraints: For example, Hardware limitations and standard compliance,
 - Software System Attributes: Such as Security, Availability, and Reliability, etc.
4. Formal Analysis using Alloy: This section will present the formal model of the system. It will include the objective of the formal modelling, a description of the model itself, what can be verified, and the results of the assertion checks or model instances.
5. Effort Spent: This section will contain a report detailing the number of hours each group member has worked to complete this document.
6. Reference: This final section lists any external documents or standards cited within this specification.

2 Overall Description

2.1 Product Perspective

Best Bike Paths (BBP) is a multi layer information system that integrates user contributed geographic data with mobile sensing and real time path computation. BBP interacts with:

1. Mobile devices for GPS, accelerometer, gyroscope, and speed data.
2. External meteorological APIs for environmental data enrichment.
3. Internal geographic and path processing modules for route reconstruction and scoring.
4. A shared user contribution database, which aggregates, merges, and updates public bike path information.

BBP maintains persistent data about bike paths, obstacle reports, user trips, and path contributions. Only registered users can insert new data, but all users, including unregistered ones, can browse the public inventory and perform path queries.

BBP handles manual user input as well as automated acquisition from sensors. It reconstructs paths from GPS traces, detects possible obstacles from irregular motion readings, validates data with the user, and publishes confirmed contributions.

The following subsections describe scenarios that highlight the shared phenomena at the interface between users, mobile devices, external APIs, and the BBP system.

2.1.1 Scenarios and Shared Phenomena

Scenario A: Manual Path Contribution

Flow

1. A registered user selects “Add New Path”.
2. User specifies the streets, segments, and the overall status of the path.
3. User optionally adds obstacles (such as potholes, road closures, etc...), specifying their location and severity.
4. BBP validates street names through its map provider and checks for duplicates.
5. User chooses either:
 - Publish immediately, or
 - Save privately for later review.

Shared Phenomena

- User sends structured metadata to BBP.
- BBP stores and later displays the path to the community.
- BBP marks the submission as manually created.

Important Notes

- Manual contributions may override or complement automatically generated ones.
- When a manual contribution contradicts earlier data, merging rules apply.

Scenario B: Automated Path Detection and Contribution

Flow

1. A registered user activates “Automatic Mode”.
2. The mobile device streams GPS coordinates, speed values, accelerometer, and gyroscope readings continuously.
3. BBP evaluates speed patterns to infer whether the user is biking.
4. Once biking is detected, BBP begins reconstructing the traveled path.
5. Sensor anomalies trigger candidate obstacle detections.
6. After stopping the acquisition, the user is asked to:
 - a. Confirm the reconstructed path shape,
 - b. Validate obstacle detections,
 - c. Provide or correct the path status.
7. User decides whether to publish the contribution.

Shared Phenomena

- Sensor readings flow from smartphone to BBP.
- System infers higher level concepts (biking, pothole, path segment).
- User validates system inferences.
- BBP stores contribution with an “automatic” source marker.

Important Notes

- Automatic detection may generate false positives.
- User validation is mandatory before publication.

Scenario C: Trip Recording and Statistics

Flow

1. User starts a trip recording session.
2. Device streams movement data.
3. BBP calculates:
 - Total distance,
 - Duration,
 - Average speed,
 - Possibly altitude changes,
 - Route shape.
4. BBP calls the external weather API, obtaining:
 - Temperature,
 - Wind speed,
 - Humidity,
 - Weather conditions.
5. Trip summary is stored and visible only to the user unless shared.

Shared Phenomena

- External API responses with environmental data.
- Sensor readings from user.
- User views processed statistics.

Scenario D: Public Path Query

Flow

1. A user enters an origin and destination.
2. BBP computes candidate paths, relying on:
 - Stored path geometries,
 - Current path status,
 - Obstacle density,
 - Overall connectivity.
3. BBP ranks paths according to a scoring model.
4. BBP visualizes scored paths on a map.

Shared Phenomena

- User inputs geospatial query parameters,
- BBP outputs scored and rendered paths.

Scenario E: Merging Conflicting Contributions

Flow

1. Multiple users contribute data about the same path segment.
2. Contributions differ in:
 - Status (optimal vs. requires maintenance),
 - Number and type of obstacles,
 - Timestamp.
3. BBP applies merging rules:
 - More recent contributions weigh more,
 - Multiple confirming contributions strengthen a status,
 - Outliers are downweighted but not discarded.

Shared Phenomena

- System receives conflicting metadata,
- System outputs reconciled authoritative path state.

2.1.2 Domain Model (Conceptual)

The domain includes several interacting entities. The following describes the classes and their relationships in detail, in the style of a complete domain model.

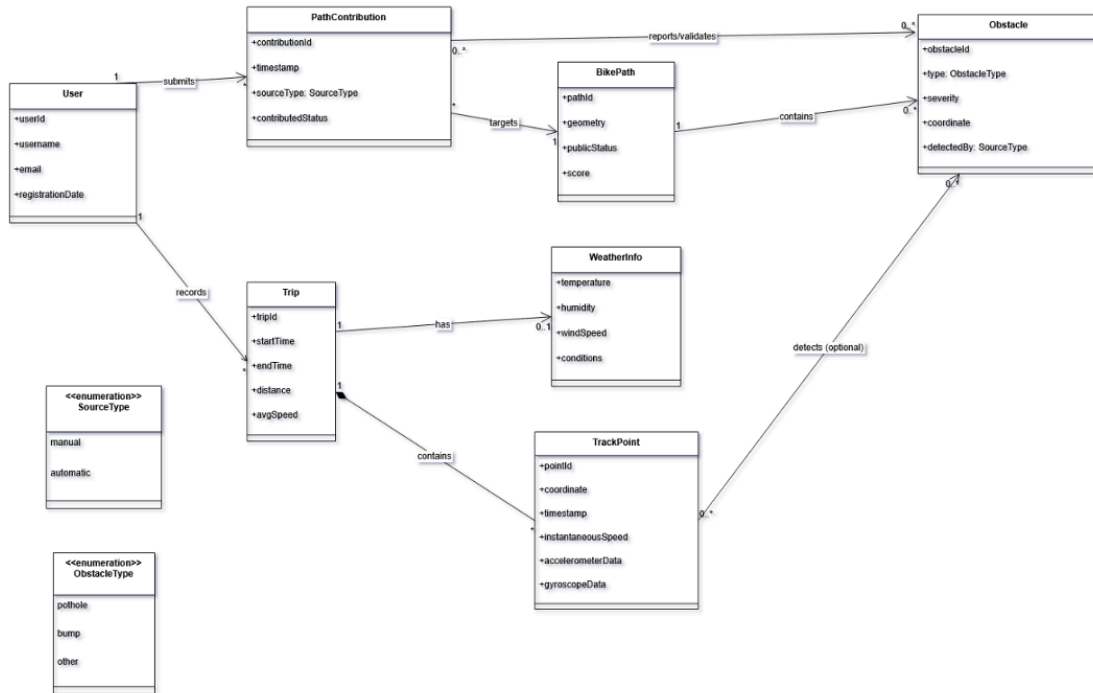


Fig: Domain Class Diagram

This Domain Class Diagram represents the main entities involved in the domain of the BBP system and the relationships between them. The diagram focuses on the conceptual structure of the system and abstracts away implementation and algorithmic details. Some important points to note:

- The diagram represents only high-level domain concepts and does not include technical components such as sensors, databases, or external APIs, which are outside the scope of the domain model.
- A User represents a registered individual interacting with the system. Guest users are not modeled explicitly, as all relevant actions (trip recording, contributions, publication) require a registered user.
- A Trip represents a single biking session recorded by a user. Each trip is composed of multiple TrackPoint instances, which represent sampled sensor and location data collected during the session.
- TrackPoint entities are modeled as part of a trip and do not exist independently. They capture instantaneous information such as location, timestamp, and sensor readings.
- BikePath represents an aggregated public representation of a physical biking path. It is not created directly by users, but rather derived from one or more user contributions.
- A PathContribution represents a user-submitted report affecting a bike path, either created manually or generated through automated trip recording and user validation.
- Obstacle entities represent physical irregularities on a bike path, such as potholes or bumps. Obstacles can be detected automatically through sensor data or manually reported by users.
- WeatherInfo is associated with a trip when available and represents contextual environmental data collected at the time of recording.

- Enumeration types (e.g., source type, obstacle type) are used to constrain certain attributes to predefined sets of values, improving clarity and consistency at the domain level.

2.1.3 State Diagrams

Below are descriptions of the main state machines in the system. Full diagrams can be produced later if needed.

State Machine 1: Automated Path Acquisition

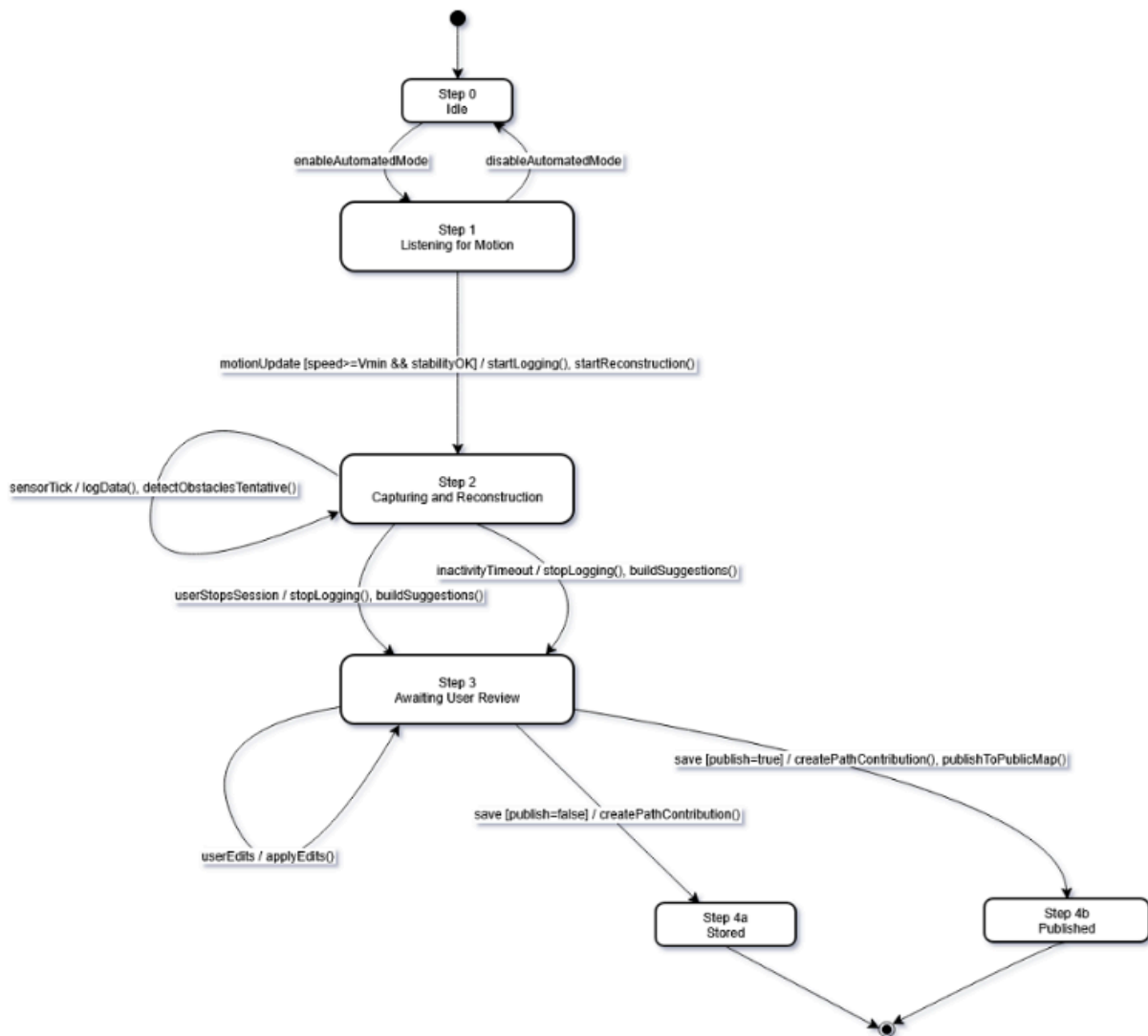


Fig: State Machine 1

This state machine describes the lifecycle of an automated path acquisition session. It models how the system detects biking activity, records sensor data, and transforms the recorded session into a validated path contribution.

Initially, the system is in an idle state, where automated mode is not active. Once the user enables automated mode, the system begins listening for motion signals. When biking activity is detected based

on speed and stability conditions, the system starts recording location and sensor data and performs path reconstruction and tentative obstacle detection.

Recording ends either when the user explicitly stops the session or when prolonged inactivity is detected. At this point, the system presents the reconstructed path, detected obstacles, and suggested path status to the user for review. The user may edit or correct the proposed information.

Finally, the user can choose to save the contribution for later use or publish it immediately. Saved contributions remain private, while published contributions become visible and are used to update the public bike path representation.

State Machine 2: Manual Contribution Lifecycle

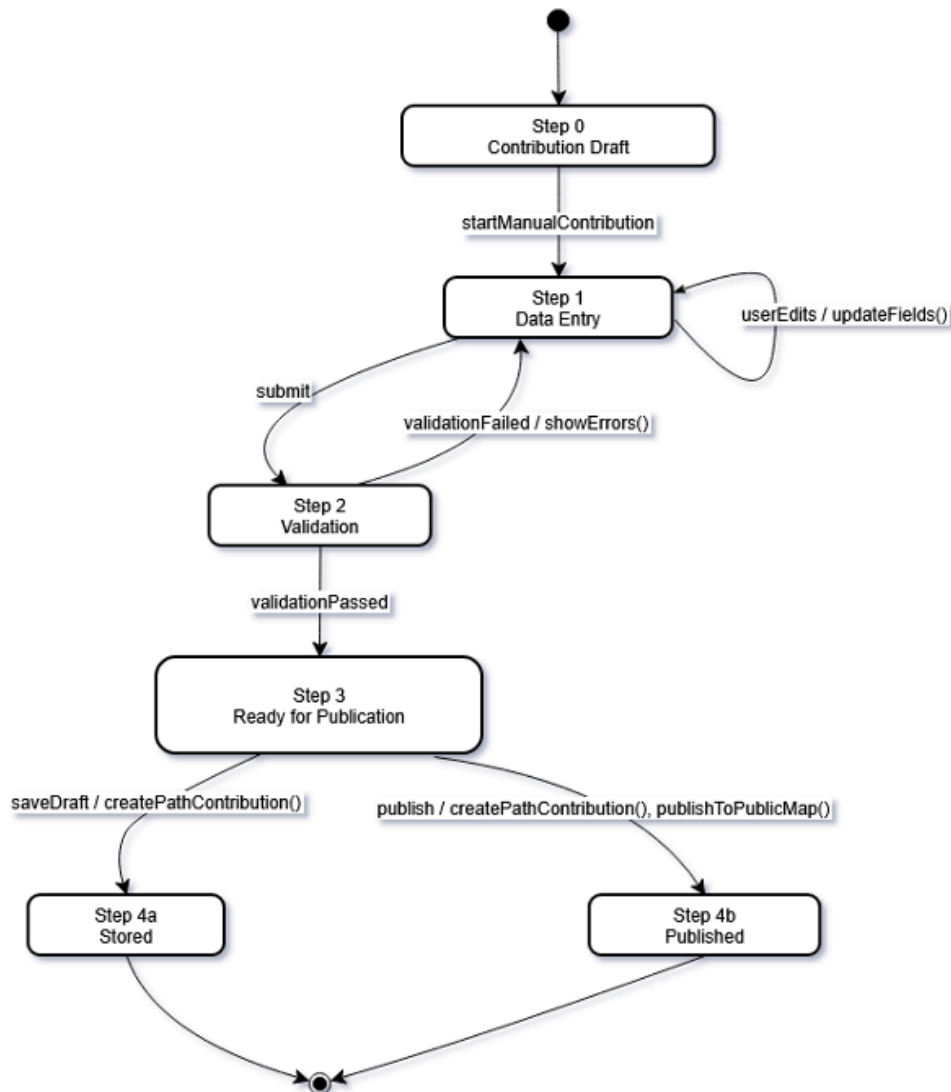


Fig: State Machine 2

This state machine represents the lifecycle of a manually created path contribution. It models the actions performed by a user when submitting information without relying on automated trip recording.

The lifecycle begins when the user initiates a manual contribution, entering a draft state. During data entry, the user specifies relevant information such as street geometry, path status, and obstacles. The

system allows iterative edits during this phase.

Once the user submits the contribution, the system performs validation checks to ensure data completeness, geometric coherence, and consistency with known street information. If validation fails, the user is returned to the editing phase with feedback.

After successful validation, the contribution becomes ready for publication. The user can either save it as a draft or publish it. Published contributions are made publicly visible and contribute to the aggregation of bike path information.

State Machine 3: Path State Aggregation

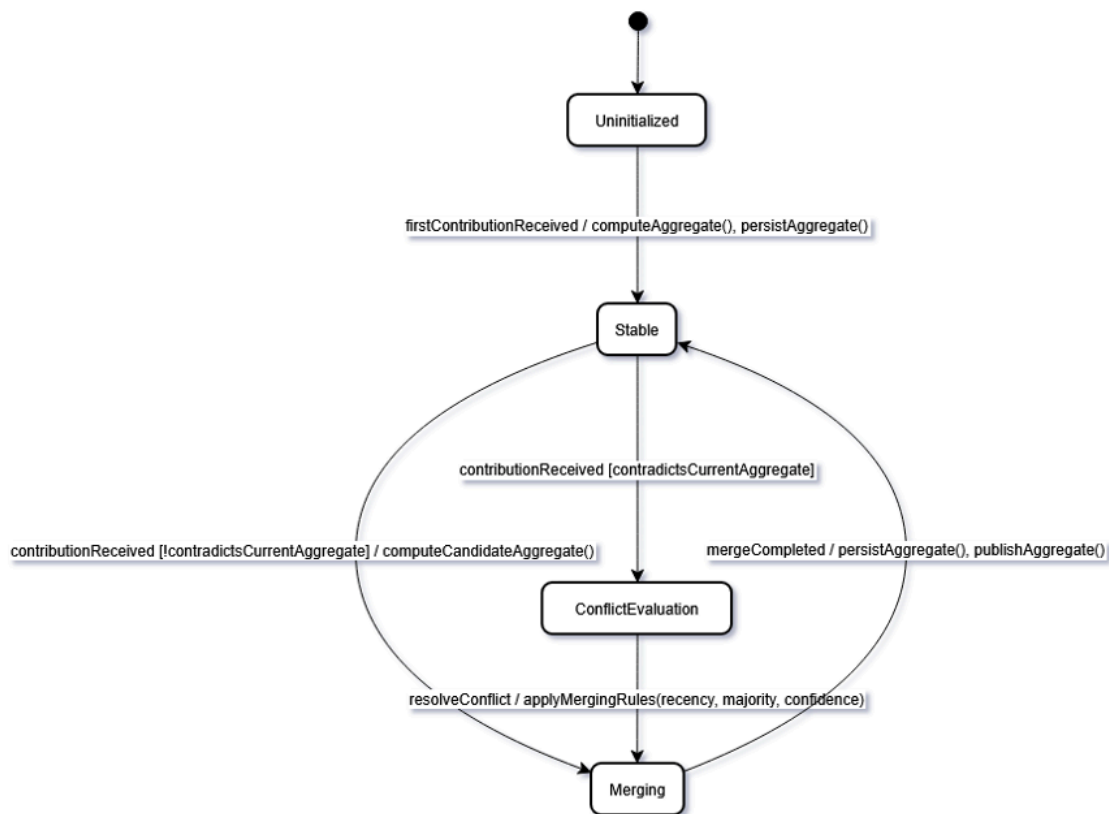


Fig: State Machine 3

This state machine describes how the system maintains and updates the aggregated public state of a bike path based on incoming contributions.

Initially, a bike path is uninitialized and has no associated public state. When the first contribution is received, the system computes an initial aggregate state, which becomes publicly visible.

Subsequent contributions may either be consistent with the existing aggregate state or contradict it. In the case of a contradiction, the system enters a conflict evaluation phase. During this phase, merging rules based on recency, majority agreement, and contribution confidence are applied to resolve inconsistencies.

Once merging is completed, the system updates and republishes the stable public state of the bike path, which serves as the authoritative representation returned by system queries.

State Machine 4: Trip Recording

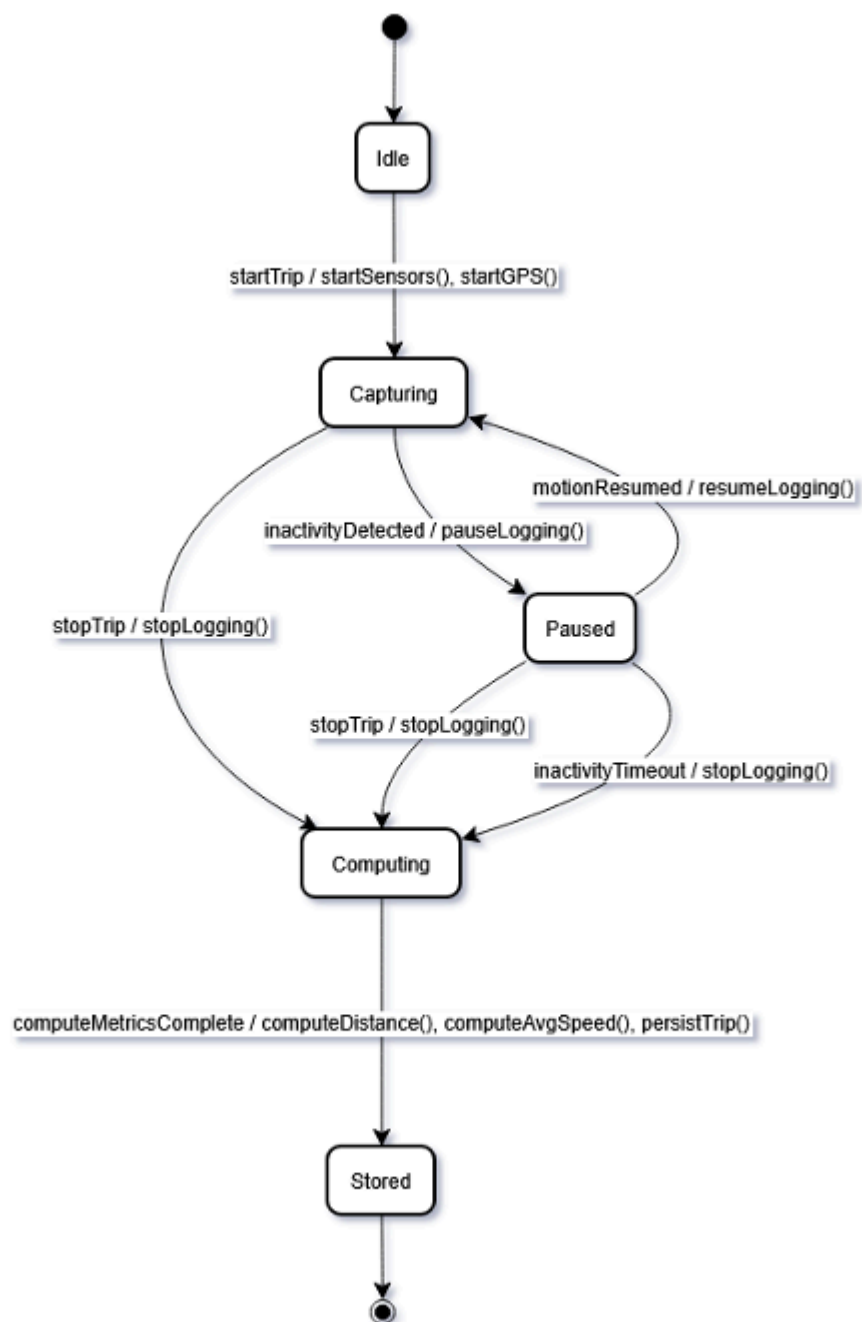


Fig: State Machine 4

This state machine models the lifecycle of a trip recorded by a user, independently of whether the trip leads to a path contribution.

The trip recording process starts in an idle state. When the user starts a trip, the system begins recording sensor and location data. Temporary inactivity during the trip may cause the recording to enter a paused state, from which recording resumes once motion is detected again.

When the trip is stopped by the user or due to prolonged inactivity, the system computes final trip metrics such as distance and average speed. After computation, the trip is stored in the user's trip history for later viewing and analysis.

2.2 Product Functions

This section describes the major system functions, grouped logically and explained in detail, with behavior and constraints.

2.2.1 Registration and Login

Registered Users

- Create an account by providing email, username, password.
- Login requires correct credentials.
- Registered users can:
 - record trips,
 - submit path contributions,
 - use automated acquisition mode,
 - publish path data.

Unregistered Users

- No login required.
- Can:
 - browse paths,
 - view map,
 - search best routes,
 - see path statuses and obstacles.

They cannot create, edit, or publish contributions.

2.2.2 Trip Recording and Statistics

Users can:

- Start trip recording at any moment.
- View ongoing trip summary:
 - current speed,
 - distance covered,

- time elapsed.

After recording:

- System computes:
 - average speed,
 - total distance,
 - path elevation patterns(if available),
 - weather info fetched from external API.

Trips belong only to their owners and are never public unless shared manually.

2.2.3 Manual Path Contribution

Users can manually insert:

- street sequences,
- path status(optimal, medium, sufficient, requires maintenance),
- obstacle locations and types.

BBP:

- validates user supplied streets,
- stores the manual contribution,
- allows immediate publication.

2.2.4 Automated Path Contribution

Automated mode performs:

- Biking detection through speed patterns,
- Path reconstruction from GPS trace,
- Obstacle detection from accelerometer and gyroscope spikes,
- Summary presentation for user review,
- Merged publication after user confirmation.

Users:

- must confirm detected obstacles,
- may override system proposed path status,
- may discard detected anomalies.

2.2.5 Path Querying and Visualization

All users can:

- enter an origin and destination,
- request best paths.

BBP:

- computes possible routes,

- evaluates them using:
 - path status,
 - obstacle density,
 - effectiveness in reaching destination,
- ranks routes by score,
- visualizes them on a map.

2.2.6 Community Data Management

BBP merges:

- conflicting user contributions,
- obsolete data replaced by fresh contributions,
- minority reports downweighted but not discarded.

Each BikePath object has:

- a stable public status,
- aggregated obstacles,
- a composite score.

2.2.7 External Weather Service Usage

BBP contacts the weather API to:

- fetch live conditions for the user's location,
- attach weather metadata to trips,
- optionally attach weather info to path reconstruction sessions.

System requires:

- valid API key,
- reliable internet.

2.3 User Characteristics

2.3.1 Registered User

A registered user:

- can create a profile,
- can log in using credentials,
- can record trips,
- can contribute data(manual or automated),
- can publish path information,
- must review automatically detected data before publication,
- can use all available features including statistics.

Typical profile:

- frequent cyclist,
- smartphone owner,
- familiar with GPS apps.

2.3.2 Unregistered User

An unregistered user:

- does not need an account,
- can browse public path data,
- can perform routing queries,
- cannot contribute data,
- cannot record trips.

Target audience:

- casual cyclists,
- tourists,
- occasional users.

2.3.3 System Administrator

The administrator:

- monitors platform integrity,
- manages path merging rules,
- resolves data conflicts,
- maintains availability of external API configurations,
- handles user reports for inappropriate content.

2.4 Assumptions, Dependencies and Constraints

2.4.1 Regulatory and Privacy Constraints

- BBP stores sensitive data such as GPS traces and trip histories.
This must comply with:
 - EU GDPR,
 - data minimization principles,
 - user consent for sensor access.
- Weather API usage must follow provider terms.

Trip data is private unless explicitly shared

3. Specific Requirements

This section presents a detailed description of the functional and non-functional requirements of the Best Bike Paths (BBP) system. It expands on the overview introduced in Section 2 and clearly defines the system's expected behavior and quality attributes, providing a solid foundation for its design and implementation.

3.1 External Interface Requirements

This section describes the external interfaces through which users and external systems interact with the Best Bike Paths (BBP) system. It focuses on the user-facing interfaces as well as any interactions with external services, defining how information is exchanged to ensure usability, compatibility, and reliability.

3.1.1 User Interfaces

The Best Bike Paths system provides a graphical user interface accessible through both mobile devices and web browsers. The interface is designed to be intuitive and usable by users with different levels of technical expertise.

The system supports two main categories of users:

- Registered users, who can record trips, insert bike path information, and publish data
- Non-registered users, who can browse and visualize publicly available bike path information

The user interface includes:

- A map-based visualization for displaying bike paths and routes
- Forms for manual insertion of bike path information
- Interfaces for trip recording and statistics visualization
- Dedicated screens for confirming or correcting automatically detected obstacles
- Search interfaces for specifying origin and destination and visualizing ranked paths

3.1.2 Hardware Interfaces

The BBP system interacts with hardware components available on mobile devices when operating in automated mode. These include:

- GPS sensors, used to track the user's location and reconstruct traveled paths
- Accelerometers, used to detect sudden vertical movements of the device
- Gyroscopes, used to detect abnormal rotations that may indicate surface irregularities

The availability and accuracy of sensor data depend on the user's device. The system shall continue to operate even if some sensors are unavailable, limiting only the affected functionalities.

3.1.3 Software Interfaces

BBP interacts with several external software systems:

- A map service to support route computation, geocoding, and visualization
- A meteorological service API to retrieve weather information such as temperature, wind speed, and conditions

- Mobile operating system APIs to access sensor data
- A database management system (DBMS) to store users, trips, and bike path information

3.1.4 Communication Interfaces

The system communicates using standard web technologies. All communications between clients and servers occur over HTTPS to ensure data confidentiality and integrity. Data exchange follows a RESTful architecture and uses JSON as the primary data format. The system may temporarily store data locally in case of intermittent connectivity.

3.2 Functional Requirements

This section defines the functional requirements of the Best Bike Paths (BBP) system. It describes the services and behaviors the system must provide to support user interactions, data management, and core functionalities, serving as a reference for system design and implementation.

3.2.1 Functional Requirements List

User Registration and Access

- R1: The system allows users to register.
- R2: The system allows registered users to log in.
- R3: The system allows non-registered users to access published bike path information.

Trip Recording

- R4: The system allows registered users to record personal trips.
- R5: The system collects GPS data during trip recording.
- R6: The system computes trip statistics, including total distance, duration, and average speed.
- R7: The system stores recorded trips for future visualization.

Meteorological Data Enrichment

- R8: The system retrieves meteorological data from an external service.
- R9: The system associates weather data with recorded trips when available.

Manual Insertion of Bike Path Information

- R10: The system allows registered users to manually insert bike path information.
- R11: The user specifies the streets composing the path.
- R12: The user specifies the path status.
- R13: The user specifies known obstacles.
- R14: The user chooses whether the information is publishable.

Automated Insertion of Bike Path Information

- R15: The system detects whether the user is biking based on speed.
- R16: The system automatically reconstructs the following path using GPS data.
- R17: The system collects accelerometer and gyroscope data.
- R18: The system detects potential obstacles based on sensor data.

Confirmation of Detected Information

- R19: The system presents detected obstacles to the user.
- R20: The user can confirm, modify, or discard detected obstacles.
- R21: Only confirmed information can be published.

Information Publishing and Merging

- R22: The system allows users to publish confirmed information.

- R23: The system merges information from multiple users regarding the same path.
- R24: The merging process considers:
 - Data freshness
 - Number of confirmations
 - Conflicting reports
- R25: The system computes a consolidated path status.

Path Search and Visualization

- R26: The system allows users to specify an origin and a destination.
- R27: The system computes possible bike paths between the two points.
- R28: The system assigns a score to each path.
- R29: The score depends on path status and path efficiency.
- R30: The system visualizes paths ordered by score.

3.2.2 Use Case Diagram

The Use Case Diagram illustrates the main interactions between users and the BBP system. It identifies the different actors involved: registered users, non-registered users, and external services and the set of use cases supported by the system. External services, such as the map service and the weather service, are modeled as secondary actors supporting specific system functionalities.

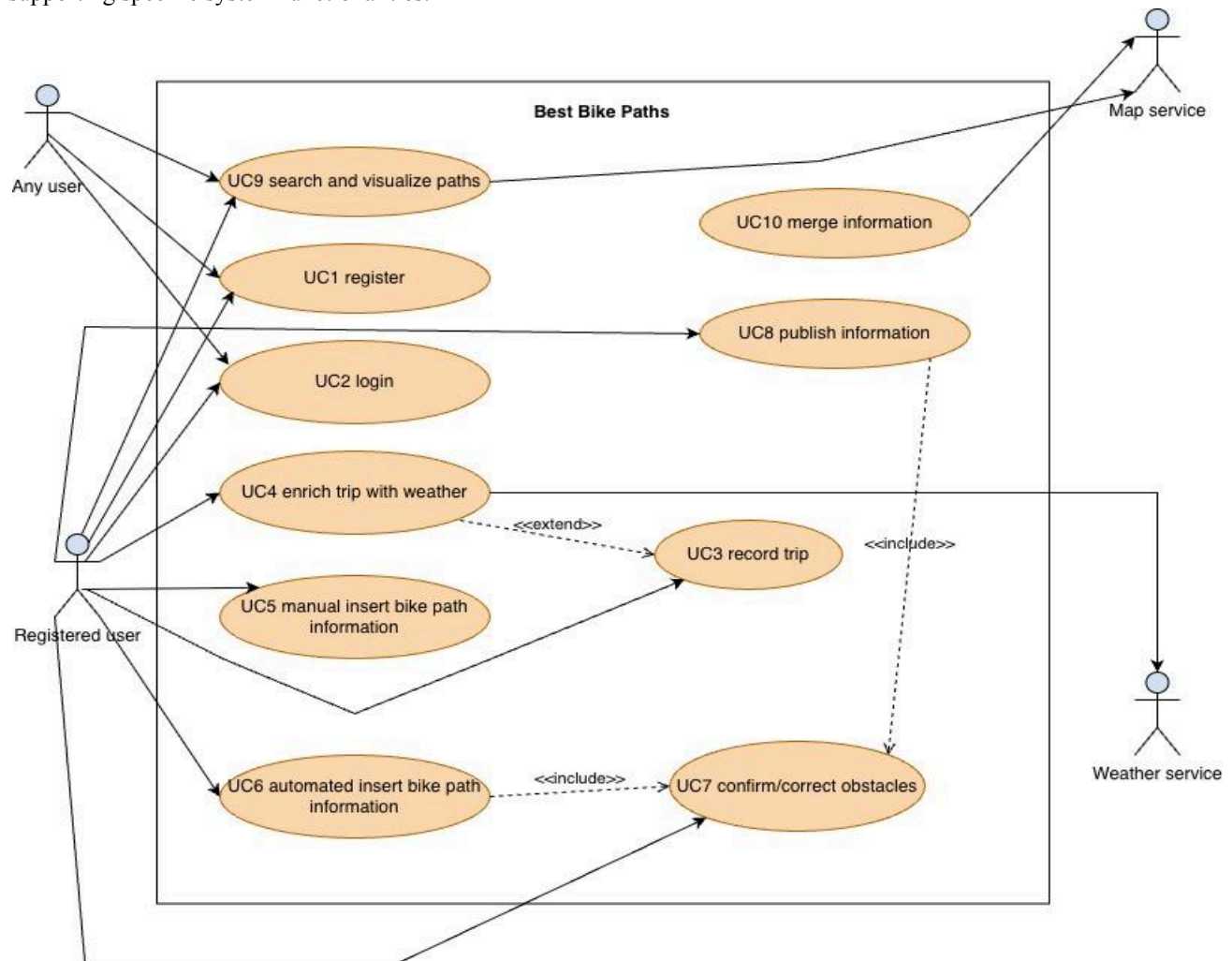


Figure 3.1: Use case diagram of Best Bike paths

3.2.3 Use Cases

The following use cases describe the system behavior from the user's perspective. Each use case is identified by a unique identifier (UCX), which is consistently referenced throughout this document.

Use case 1 : User registration

Field	Description
Actors	Visitor
Entry condition	The user accesses the BBP application
Event flow	(a) The user selects the registration option. (b) The system displays a registration form. (c) The user submits the required information. (d) The system validates the data and creates a new account.
Exit condition	A registered user account is created
Exceptions	Invalid or missing data (user already registered)

Use case 2 : User login

Field	Description
Actors	Registered user
Entry condition	The user has a registered account
Event Flow	(a) The user enters credentials. (b) The system validates them. (c) The user is authenticated and logged in.
Exit Condition	User session is active
Exceptions	Invalid credentials

Use case 3 : Record personal trip

Field	Description
Actors	Registered User
Entry Condition	User is logged in
Event Flow	(a) User starts trip recording. (b) System collects GPS samples. (c) User stops recording. (d) The system computes trip statistics and stores the trip.
Exit Condition	Trip is stored with statistics
Exceptions	GPS unavailable

Use case 4 : Enrich trip with weather data

Field	Description
Actors	Registered User, Weather Service
Entry Condition	A recorded trip exists

Event Flow	(a) The system requests weather data for the trip. (b) Weather service returns data. (c) The system associates weather data with the trip.
Exit Condition	Trip is enriched with meteorological information
Exceptions	Weather service unavailable

Use case 5 : Insert Bike Path Information (Manual Mode)

Field	Description
Actors	Registered User
Entry Condition	User is logged in
Event Flow	(a) User enters path streets and status. (b) The user optionally adds obstacles. (c) The user saves the information as private or publishable.
Exit Condition	Path information is stored
Exceptions	Missing mandatory fields

Use case 6: Insert Bike Path Information (Automated Mode)

Field	Description
Actors	Registered User
Entry Condition	User is logged in (sensors available)
Event Flow	(a) System detects biking activity. (b) GPS data is collected. (c) Sensors detect anomalies. (d) Draft path information is generated.
Exit Condition	Draft submission is created
Exceptions	Sensor data unavailable

Use case 7: Confirm or Correct Detected Obstacles

Field	Description
Actors	Registered User
Entry Condition	Draft submission exists
Event Flow	(a) The system displays detected obstacles. (b) User confirms, edits, or removes obstacles. (c) User saves confirmed data.
Exit Condition	Obstacles are confirmed
Exceptions	User exits without saving

Use case 8 : Publish Bike Path Information

Field	Description
-------	-------------

Actors	Registered User
Entry condition	Confirmed path information exists
Event flow	(a) User selects publish option. (b) System validates and publishes information.
Exit condition	Information is publicly visible
Exceptions	Information not confirmed

Use case 9 : Search and Visualize Bike Paths

Field	Description
Actors	Any User, Map Service
Entry condition	User accesses search interface
Event flow	(a) User specifies origin and destination. (b) System computes routes. (c) Paths are ranked and displayed.
Exit condition	Ranked paths are shown
Exceptions	Invalid locations

Use case 10 : Merge Bike Path Information

Field	Description
Actors	BBP System
Entry condition	Multiple publishable submissions exist
Event flow	(a) System aggregates data. (b) Conflicts are resolved using freshness and confirmations.
Exit condition	Consolidated path information is updated
Exceptions	Inconsistent data

3.2.4 Sequence Diagrams

Sequence diagrams show how users interact with the BBP system in different situations. The figures below present the main scenarios, such as recording a trip, automatically adding data with confirmation, searching and viewing bike paths, and merging information. Each diagram shows how messages are exchanged between the system and external services.

SD1 : Record trip (UC3) + optional weather (UC4)

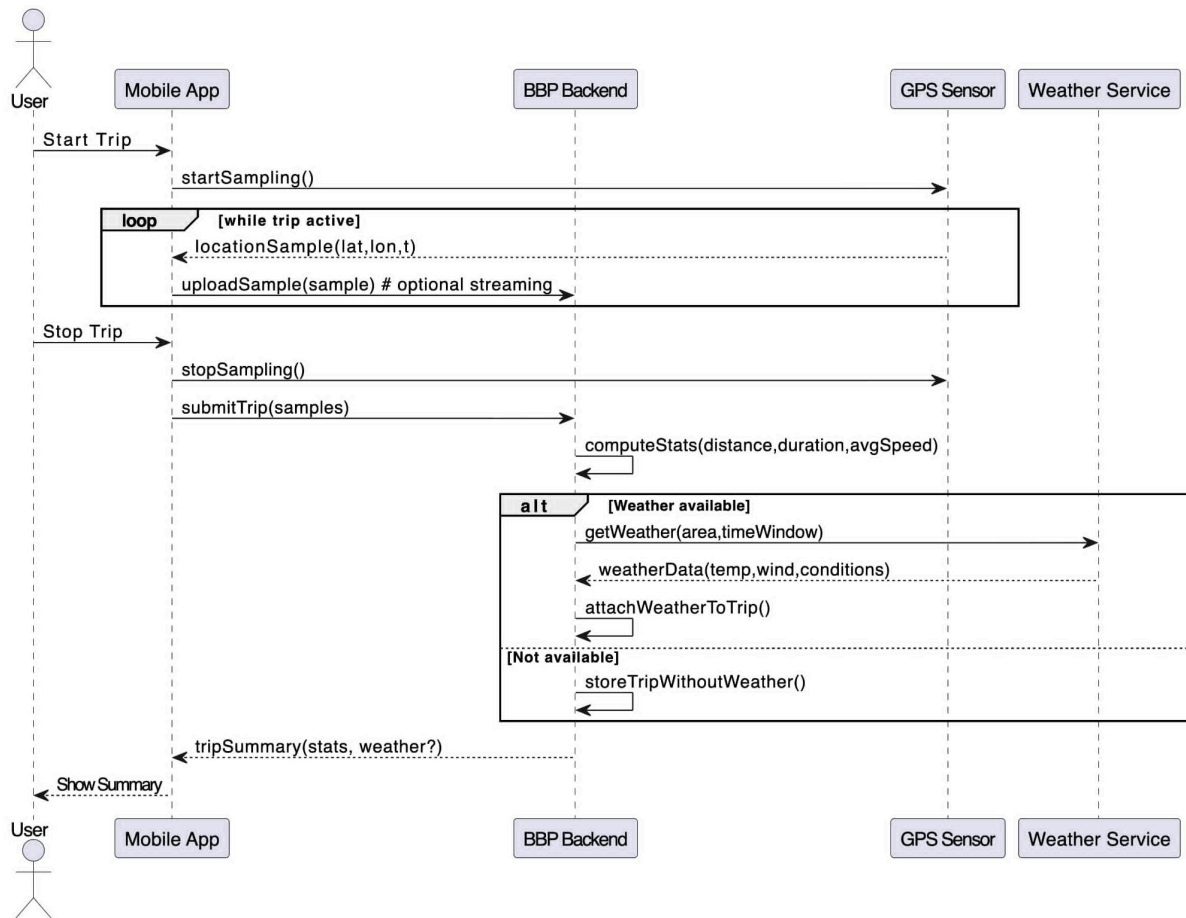


Figure 3.2 : Record trip and optional weather

SD2 : Manual insert bike path information (UC5)

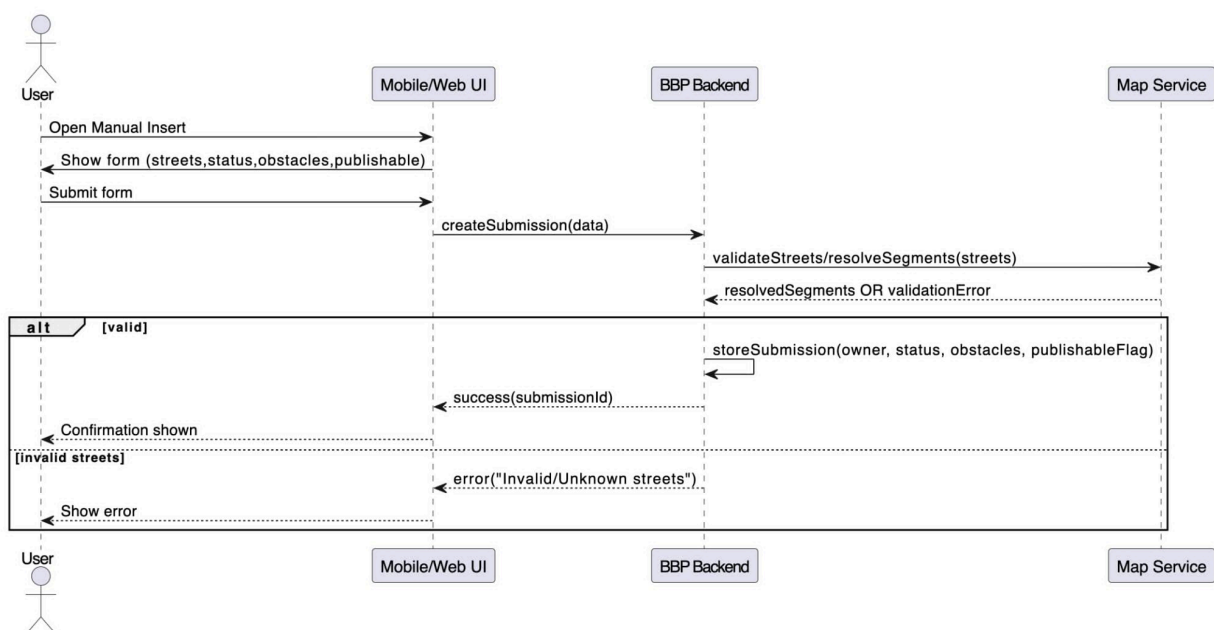


Figure 3.3 : Manual insert bike path information

SD3 : Automated insert + Detect obstacles (UC6)

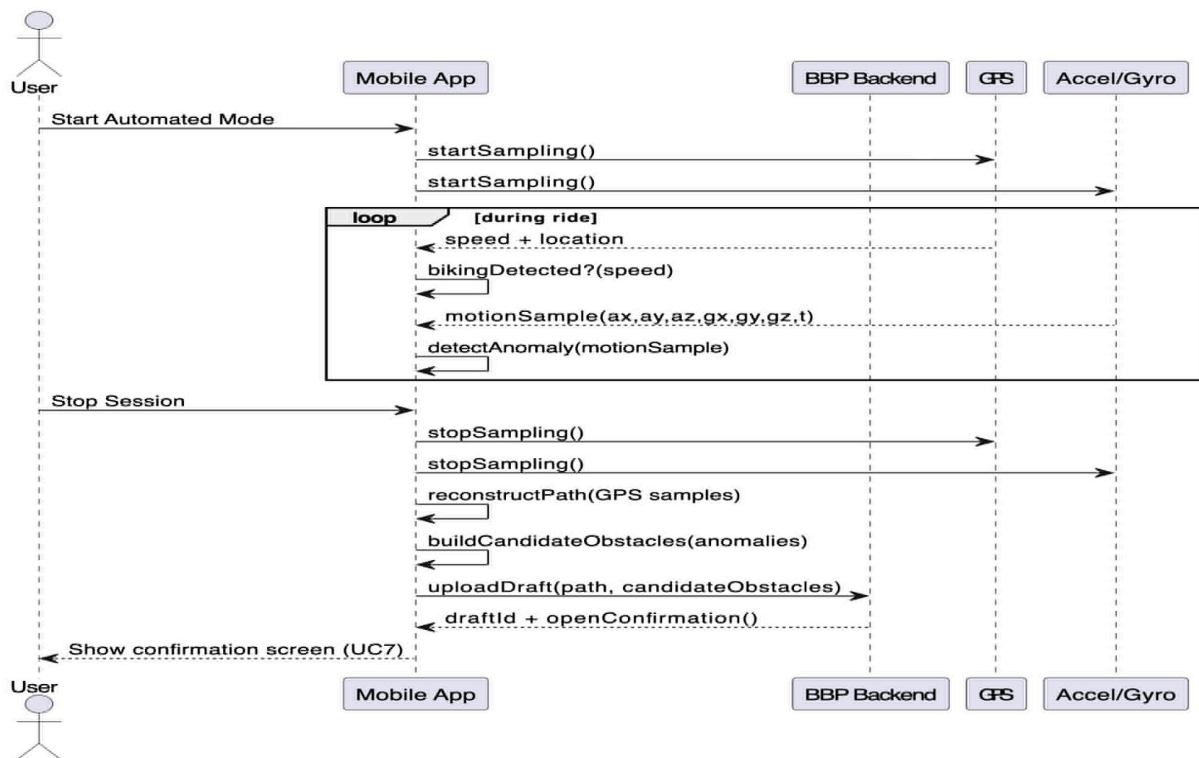


Figure 3.4: Automated insert and detect obstacles

SD4 : Confirm / Correct obstacles (UC7) then publish (UC8)

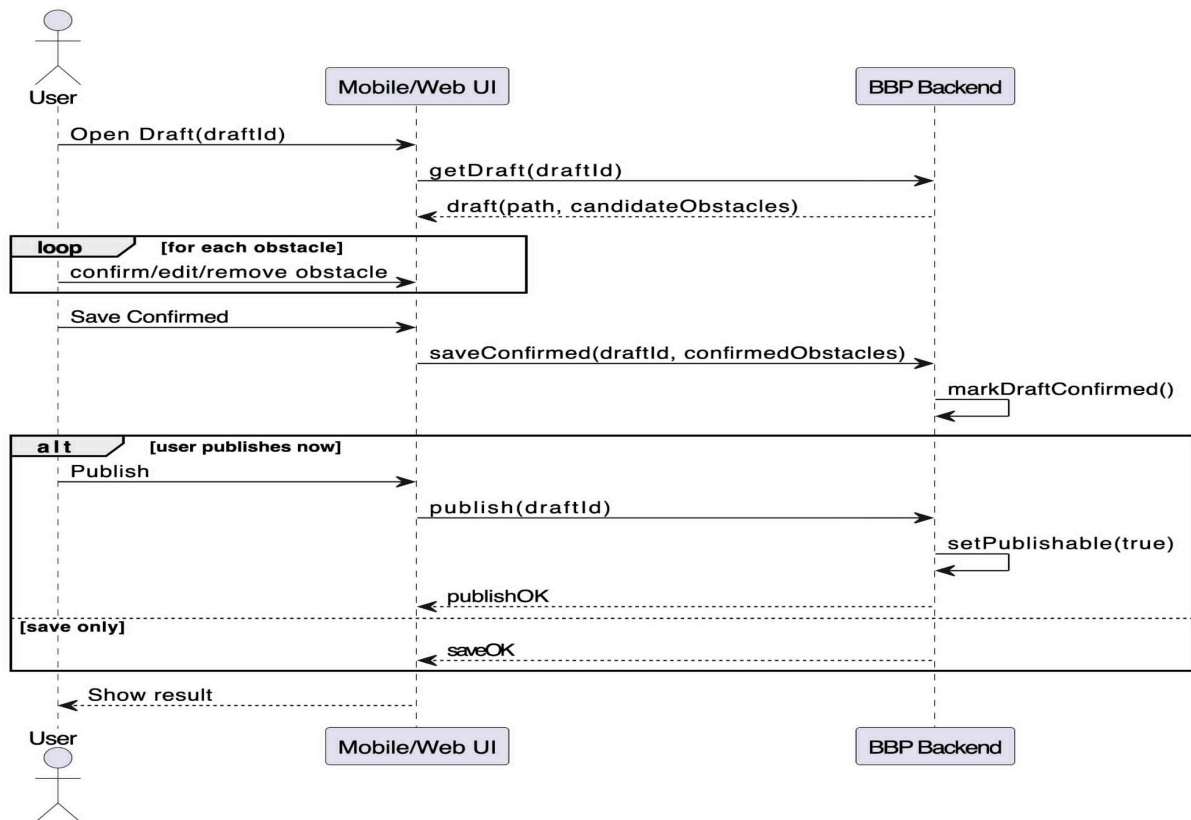


Figure 3.5: Confirm and Correct obstacles

SD5: Search and visualize paths (UC9) with scoring

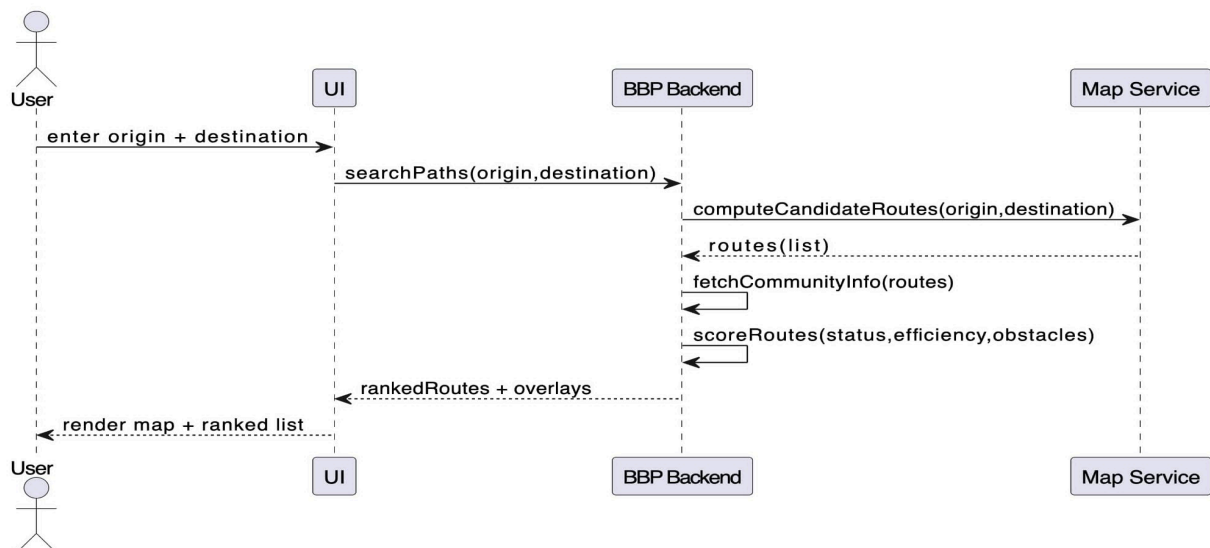


Figure 3.6: Search and visualize paths

SD6: Merge information (UC10) (Background job)

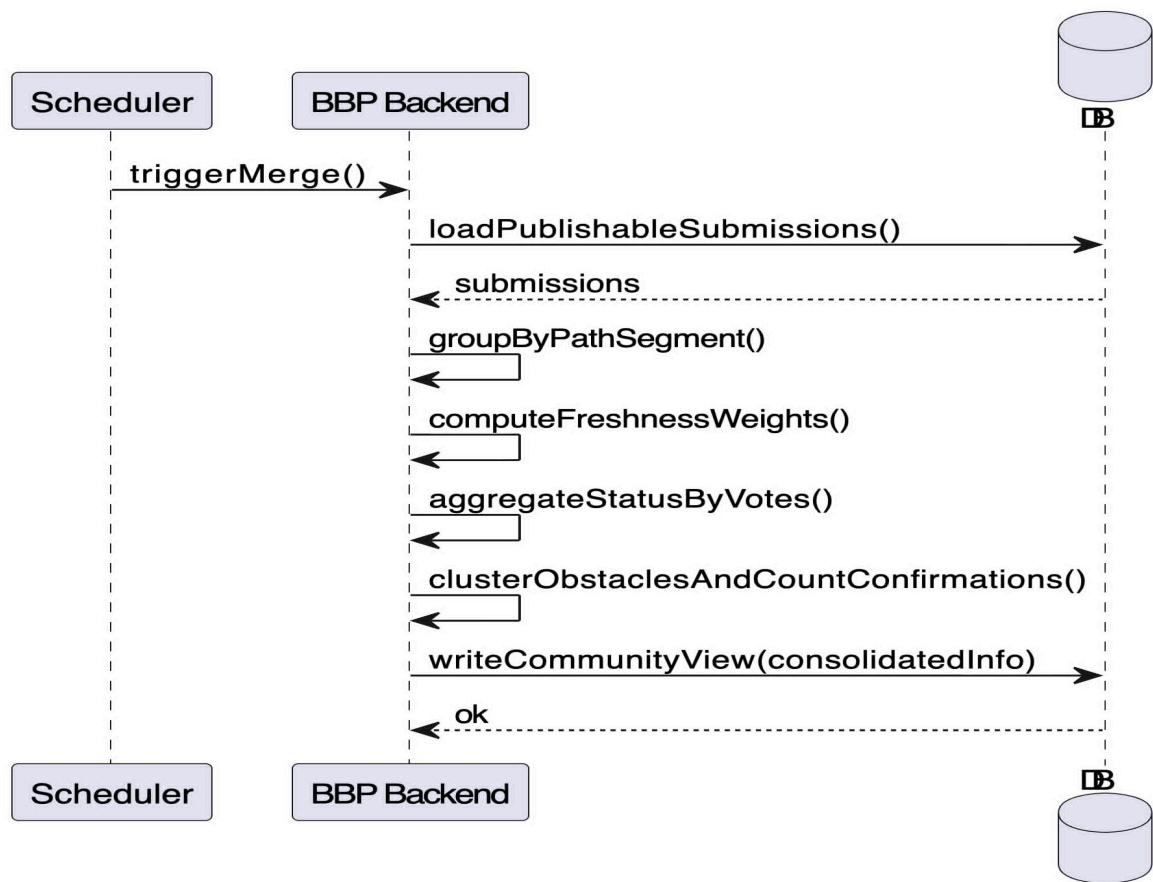


Figure 3.7: Merge Information

3.2.5 Requirement Mapping

The purpose of this mapping is to ensure that each system goal is adequately supported by one or more functional requirements and relies on clearly stated assumptions about the domain.

Goal G1

Support users in recording and analyzing their personal cycling activities.

Mapped Requirements:

- R4: The system allows registered users to record personal trips.
- R5: The system collects GPS data during trip recording.
- R6: The system computes trip statistics.
- R7: The system stores recorded trips for future visualization.
- R8: The system retrieves meteorological data from an external service.
- R9: The system associates weather data with recorded trips when available.

Related Domain Assumptions:

- A1: Users carry a mobile device with GPS capabilities while biking.
- A2: The GPS data provided by mobile devices is sufficiently accurate.
- A3: External meteorological services provide reliable data when available.

Goal G2

Enable registered users to insert and publish reliable information about bike paths.

Mapped Requirements:

- R10: The system allows registered users to manually insert bike path information.
- R11: The user specifies the streets composing the path.
- R12: The user specifies the path status.
- R13: The user specifies known obstacles.
- R14: The user chooses whether the information is publishable.
- R22: The system allows users to publish confirmed information.

Related Domain Assumptions:

- A4: Users provide truthful and accurate information.
- A5: Streets and paths specified by users exist in the map service database.

Goal G3

Support automated acquisition of bike path information and ensure its validation before publication.

Mapped Requirements:

- R15: The system detects whether the user is biking based on speed.
- R16: The system automatically reconstructs the following path using GPS data.
- R17: The system collects accelerometer and gyroscope data.
- R18: The system detects potential obstacles based on sensor data.
- R19: The system presents detected obstacles to the user.
- R20: The user can confirm, modify, or discard detected obstacles.
- R21: Only confirmed information can be published.

Related Domain Assumptions:

- A6: Sensor data reflects real-world road conditions with acceptable accuracy.
- A7: Users actively review and validate automatically detected information.

Goal G4

Provide users with ranked and informative bike path suggestions between two locations.

Mapped Requirements:

- R3: The system allows non-registered users to access published bike path information.
- R26: The system allows users to specify an origin and a destination.
- R27: The system computes possible bike paths between the two points.
- R28: The system assigns a score to each path.
- R29: The score depends on path status and path efficiency.
- R30: The system visualizes paths ordered by score.

Related Domain Assumptions:

- A8: Map services correctly compute feasible bike paths.
- A9: Published path information is sufficiently available to compute scores.

Goal G5

Ensure that bike path information collected from multiple users is consistent and up to date.

Mapped Requirements:

- R23: The system merges information from multiple users regarding the same path.
- R24: The merging process considers data freshness, number of confirmations, and conflicting reports.
- R25: The system computes a consolidated path status.

Related Domain Assumptions:

- A10: Multiple users may provide overlapping information for the same path.
- A11: More recent and frequently confirmed information is more reliable.

3.3 Performance Requirements

- The system shall return path search results within **3 seconds** under normal load.
- Sensor data acquisition shall not significantly affect device performance.
- The system shall support multiple concurrent users.

3.4 Design Constraints

This section describes the constraints that influence the design and implementation of the Best Bike Paths (BBP) system. These constraints include regulatory requirements, technical limitations, and external dependencies that must be respected to ensure correct and reliable system behavior.

3.4.1 Standards Compliance

- The system shall comply with **GDPR** regulations.
- REST APIs shall follow standard HTTP conventions.

3.4.2 Hardware Limitations

- Sensor precision depends on device hardware.
- Battery consumption shall be minimized during automated mode.

3.4.3 Other Constraints

- Availability of meteorological data depends on third-party services.
- Map service rate limits may apply.

3.5 Software System Attributes

This section describes the quality attributes of the Best Bike Paths (BBP) system. It defines non-functional characteristics such as reliability, availability, security, maintainability, and portability that the system must satisfy to ensure a robust and high-quality solution.

3.5.1 Reliability

The system shall ensure data consistency and prevent loss of collected information.

3.5.2 Availability

The system shall guarantee high availability, with a target uptime of **99.9%**.

3.5.3 Security

- All data in transit shall be encrypted.
- User data shall be securely stored.

3.5.4 Maintainability

The system shall follow modular design principles and be well documented.

3.5.5 Portability

BBP shall be accessible on different devices and operating systems.

4. Formal Analysis Using Alloy

In this section, we present to you the formal analysis of the Best Bike Paths (BBP) using Alloy 6.

The purpose of this analysis is to verify the correctness of the data formally used by BBP when collecting information about bike path conditions from multiple users, both in manual and automated modes.

In particular, the formal modeling activity is driven by the following objectives:

- To model how reports on bike path segments evolve.
- To ensure that unconfirmed automated reports do not affect public information.
- To verify that fresh information prevails over stale data.
- To prove that the aggregation of reports is deterministic and safe, even in the presence of conflicting data.

To achieve this, we model Segments as the basic units of bike paths, and Reports as mutable entities whose confirmation and freshness evolve. The formal model allows us to reason about both safety properties (something bad never happens) and liveness properties (something good eventually happens).

4.1 Signatures and Domain Modeling

We start by defining the static entities of the system. A Segment represents an atomic portion of a bike path and has a mutable status that changes over time based on user reports. We present the signatures and domain modeling.

```
module BestBikePaths

open util/boolean

enum StatusValue { Optimal, Medium, Bad, MaintenanceRequired }
enum SourceType { Manual, Automated }

sig Location {}

sig Segment {
    startNode: one Location,
    endNode: one Location,
    var currentStatus: one StatusValue
}

sig User {}
```

A Report represents a piece of information provided by a user about a segment. Reports are mutable, since their freshness and confirmation status evolve.

```
sig Report {  
    target: one Segment,  
    author: one User,  
    declaredValue: one StatusValue,  
    source: one SourceType,  
    var isFresh: one Bool,  
    var isConfirmed: one Bool  
}
```

4.2 Structural Constraints

We impose basic constraints on the topology and on the lifecycle of reports.

```
fact GraphTopology {  
    all s: Segment | s.startNode != s.endNode  
}
```

This ensures that a segment always connects two distinct locations.

4.3 Valid Reports and Consensus Logic

Only **fresh and confirmed reports** are allowed to influence the public status of a segment.

```
fun validReports[s: Segment]: set Report {  
    { r: Report | r.target = s and r.isFresh = True and r.isConfirmed = True }  
}
```

The following predicate defines how the system updates the status of a segment based on the valid reports.

The decision is made using a majority rule, with a pessimistic tie-breaking strategy to favor safety.

```
pred updateSegmentStatus[s: Segment] {  
    let vr = validReports[s] |  
    let opt = #{r: vr | r.declaredValue = Optimal} |  
    let bad = #{r: vr | r.declaredValue = Bad} |  
    let maint = #{r: vr | r.declaredValue = MaintenanceRequired} |
```

```

    (maint > opt and maint > bad) implies s.currentStatus' = MaintenanceRequired
  else (bad > opt) implies s.currentStatus' = Bad
  else (opt >= bad and opt >= maint and #vr > 0) implies s.currentStatus' = Optimal
  else s.currentStatus' = Medium
}

```

4.4 Modeling System Events

We model the main system events that affect reports.

Manual report submission

```

pred submitManualReport[r: Report] {
  r.source = Manual
  r.isFresh = True
  r.isConfirmed = True
}

```

Automated report detection

```

pred automatedDetection[r: Report] {
  r.source = Automated
  r.isFresh = True
  r.isConfirmed = False
  r.declaredValue = Bad
}

```

User confirmation of automated reports

```

pred confirmReport[r: Report] {
  r.source = Automated
  r.isConfirmed = False
  r.isConfirmed' = True
}

```

Freshness decays over time

```
fact FreshnessDecay {
    always (all r: Report | r.isFresh = True implies eventually (r.isFresh = False))
}
```

This is the initial world of the alloy model. Ordering relations shown in the visualizer are Alloy-internal and not relevant to the domain model.

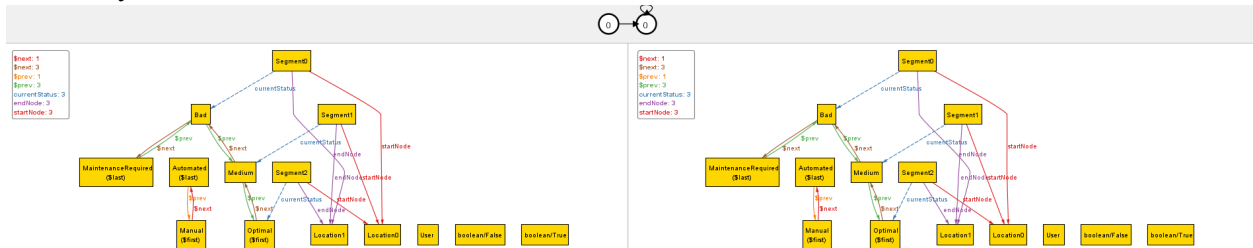


Fig: Initial Alloy Model

4.5 Example Worlds (Model Simulation)

Example 1 – Manual vs Automated Reports

This example shows how unconfirmed automated reports do not affect the segment status.

```
pred showConsensusExample {
  some r1, r2: Report |
    r1.source = Automated and r1.isConfirmed = False and
    r2.source = Manual and r2.isConfirmed = True
}

run showConsensusExample for 5
```

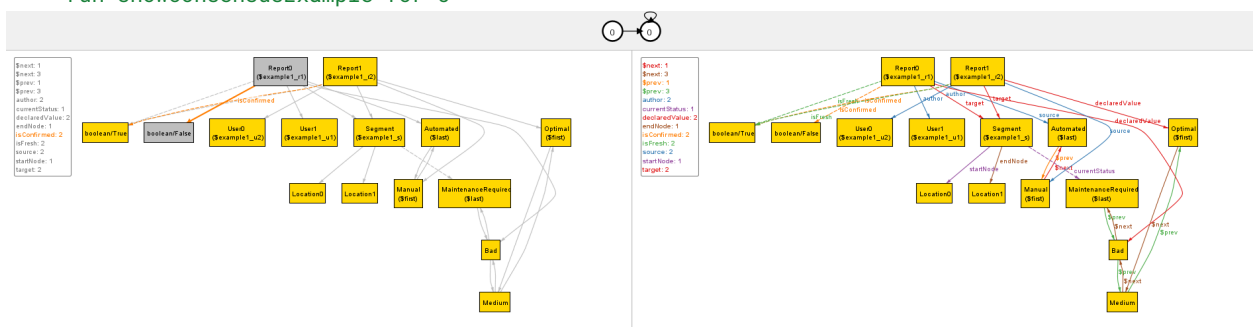


Fig: Manual vs Automated Report

Example 2 – Conflicting Reports and Tie-Breaking

```
pred conflictingReports {
  some s: Segment |
  some r1, r2: Report |
```

```

r1.target = s and r2.target = s and

r1.declaredValue = Optimal and

r2.declaredValue = Bad and

r1.isConfirmed = True and

r2.isConfirmed = True

}

run conflictingReports for 5

```

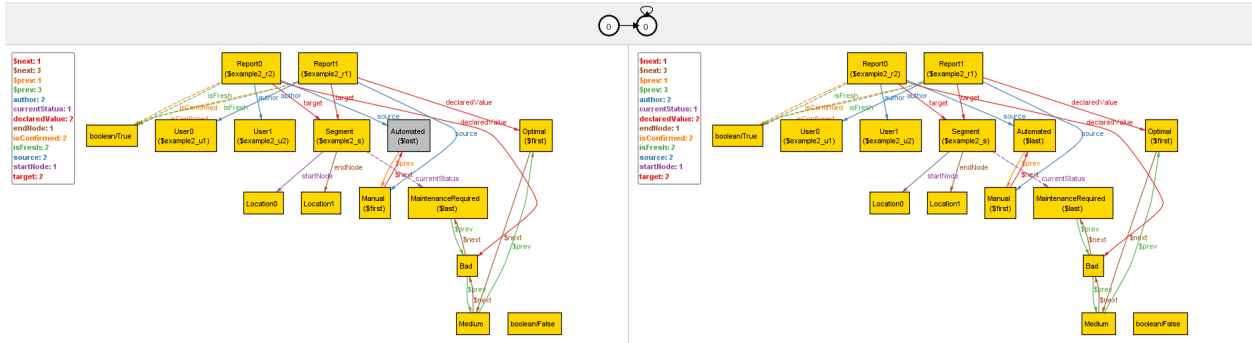


Fig: Conflicting reports and Tie-Breaking

4.6 Assertions and Verification

Safety Property: Unconfirmed Reports Never Affect Status

```

assert NoUnconfirmedInfluence {
  always (
    all s: Segment |
      no r: validReports[s] | r.isConfirmed = False
  )
}

check NoUnconfirmedInfluence for 8

```

Executing "Check NoUnconfirmedInfluence for 8"

Actual scopes: exactly 4 StatusValue, exactly 1 Optimal, exactly 1 Medium, exactly 1 Bad, exact
 Solver=sat4j Steps=1.10 Bitwidth=4 MaxSeq=7 SkolemDepth=1 Symmetry=20 Mode=batch
 1.10 steps. 47493 vars. 5790 primary vars. 78372 clauses. 215ms.
 No counterexample found. Assertion may be valid. 3ms.

Fig: No Counterexample on NoUnconfirmedInfluence

Freshness Property: Fresh Data Overrides Old Data

```

assert FreshOverridesStale {
  always (
    all s: Segment |
      (some r: validReports[s] | r.declaredValue = Optimal)
    implies s.currentStatus = Optimal
  )
}

check FreshOverridesStale for 8

```

Executing "Check FreshOverridesStale for 8"

Actual scopes: exactly 4 StatusValue, exactly 1 Optimal, exactly 1 Medium, exactly 1 Bad, exactly 1 Stale
 Solver=sat4j Steps=1.10 Bitwidth=4 MaxSeq=7 SkolemDepth=1 Symmetry=20 Mode=batch
 1.10 steps. 50271 vars. 5735 primary vars. 79174 clauses. 248ms.
 No counterexample found. Assertion may be valid. 13ms.

Fig: No CounterExample on Overriding

4.7 Importance of the Results

The formal analysis proves that:

- Automated reports cannot pollute the system unless confirmed.
- Old information naturally expires and does not dominate future decisions.
- Conflicting reports are handled in a safe and deterministic manner.
- The BBP merging logic is sound, stable, and aligned with the problem domain.

This confirms that the requirements related to data aggregation, freshness, and confirmation are not only intuitively correct but also formally verifiable.

5. Effort Spent

Group Member	Detail of Work	Hours Spent
Nafis Ismam	Initial Setup & Scope: Introduction, definition of the project's scope, and an analysis of the relevant phenomena. Formal Modeling & Verification: Development of a formal model using Alloy, including the definition of assertions and the verification process. Implementation & Execution: Translation of the model into code, execution of the code, and management via the GitHub repository (including merging). Final Documentation: Compilation of the final document, including a list of references and an effort report.	22
Ayman Mansur	Specific Requirements & Diagrams: Definition of the system's functional and non-functional requirements, external interfaces, requirement mapping, and creation of UML use case and sequence diagrams.	24
Chris Khoury	Overall Description & System Modeling: Development of the product perspective, scenarios, domain model description, state machines, user characteristics, and system assumptions and constraints.	20

References

- [1] S. Bernardi, J. Merseguer, and D. C. Petriu. A dependability profile within MARTE. *Software and Systems Modeling*, 10(3):313–336, 2011.
- [2] S. Weikl and P. Mayer, “Data-driven quality assessment of cycling networks,” *Front. Future Transp.*, vol. 4, Mar. 2023, Art. 1127742, doi: 10.3389/ffutr.2023.1127742.
- [3] T. Ahmed, A. Pirdavani, G. Wets, D. Janssens, “Assessment methods for bicycle environment safety and comfort: A scoping review,” *Sustainable Futures*, 2025, Art. 100457, doi:10.1016/j.sftr.2025.100457.
- [4] A.-S. Karakaya, L. Thomas, D. Koljada, and D. Bermbach, “A Crowdsensing Approach for Deriving Surface Quality of Cycling Infrastructure,” *arXiv*, May 2023.
- [5] A. Vybornova, T. Cunha, A. Gühnemann, and M. Szell, “Automated Detection of Missing Links in Bicycle Networks,” *arXiv*, Jan. 2022.
- [6] (Report) Benchmarking Bike Networks: Summary & Best Practices for Safer Cycling Infrastructure, League of American Bicyclists.
- [7] D. Jackson, “Alloy: A Lightweight Object Modelling Notation,” *ACM Trans. Softw. Eng. Methodol.*, vol. 11, no. 2, pp. 256–290, Apr. 2002.
- [8] N. Macedo, A. Cunha, J. Pereira, R. Carvalho, R. Silva, A. C. R. Paiva, and M. Sozinho Ramalho, “Experiences on Teaching Alloy with an Automated Assessment Platform,” in *Rigorous State-Based Methods*, Springer, 2020, pp. 61–77.
- [9] A. Sree-Kumar, E. Planas, and R. Clarisó, “Analysis of Feature Models Using Alloy: A Survey,” *arXiv*, Apr. 2016.
- [10] R. Lorch, B. Meng, K. Siu, A. Moitra, and M. Durling, *Formal Methods in Requirements Engineering: Survey and Future Directions*, 2024.